

Методы векторизации вычислений в задачах суперкомпьютерного моделирования

Рыбаков Алексей Анатольевич¹⁾

¹⁾ НИЦ «Курчатовский институт»

Москва, 1-3 декабря 2025 г.

Цель исследования

Современные задачи, решаемые с помощью компьютерного моделирования, крайне требовательны к вычислительным ресурсам. Среди них решение уравнений математической физики, создание цифровых двойников, работа с расчетными сетками, оптимизационные задачи.

Для повышения эффективности использования суперкомпьютерных систем требуется реализация распараллеливания вычислений на всех уровнях:

распараллеливание **на уровне вычислительных узлов** – на распределенной памяти;

распараллеливание **на уровне потоков** – на общей памяти;

распараллеливание **на уровне инструкций** – векторизация.

При отказе от векторизации невозможно достичь производительности вычислений выше 6,25% от пиковой.

Целью исследования является разработка методов векторизации для повышения эффективности вычислений.

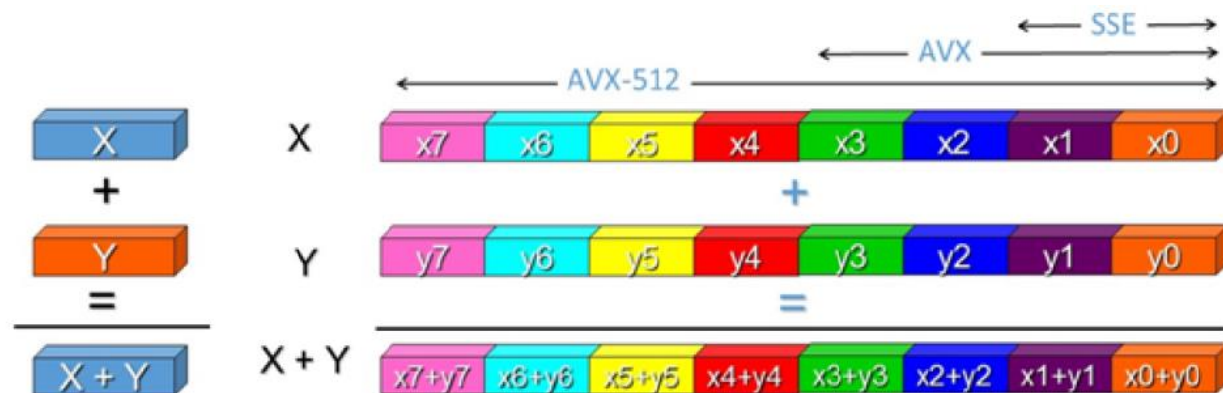


Иллюстрация векторизации вычислений

Векторизация вычислений

Векторизация – низкоуровневая оптимизация, с помощью которой можно добиться кратного повышения производительности кода.

Для оценки производительности векторного кода используются понятия:

ширина векторизации $w = \frac{v}{t}$, **ускорение** $s_{vec} = \frac{T}{T_v}$, **эффективность векторизации** $e_{vec} = \frac{s_{vec}}{w}$,
а также **логическое ускорение** $s_{vec}^* = \frac{L}{L_v}$, **логическая эффективность** $e_{vec}^* = \frac{s_{vec}^*}{w}$

(v, t – размер вектора и элемента вектора; T, T_v – времена выполнения скалярной и векторной версии; L, L_v – количество инструкций в скалярной и векторной версии).

Наиболее перспективный набор векторных инструкций AVX-512 активно развивается и поддерживан в современных микропроцессорах Intel и AMD.

Основной особенностью AVX-512, позволяющей векторизовать программный контекст с командами управления, является использование **масочных регистров**, с помощью которых обеспечивается выборочная обработка элементов векторов. Эта особенность позволяет переводить программный код, записанный в предикатной форме, в аналогичный векторный код.

Основные факторы, препятствующие эффективной векторизации – **доступ к данным** и **операции передачи управления**.

Понятие плоского цикла

В расчетных кодах часто наблюдается обработка однотипных объектов в циклах (например, ячейки расчетной сетки), отсюда возникает идея унификации векторизуемого программного контекста.

Плоским циклом будем называть цикл `for`, в котором индуктивная переменная i последовательно принимает значения от 0 до $w - 1$, где w – ширина векторизации.

```
for (int i = 0; i < w; ++i)
{
    block(i)
}
```



Требования, предъявляемые к плоским циклам:

- **обращение к данным:** на i -ой итерации все обращения к данным на запись имеют вид $a[i]$, а обращения к данным на чтение имеют вид либо $a[i]$, либо являются чтением скаляров;
- **выравнивание данных** в памяти: все массивы данных, обращение к которым на i -ой итерации имеет вид $a[i]$, выровнены в памяти на размер вектора;
- **отсутствие межитерационных зависимостей.**

Циклы, в которых нарушаются некоторые требования, будем называть **квазиплоскими**.

Исследуются методы векторизации плоских циклов с телом произвольного вида.

Семантика вещественных векторных инструкций

Имя инструкции	Семантика инструкции
VMOVAPS, VMOVUPS, VSQRTPS, VGETEXPPS, VGETMANTPS, VRCP14PS, VREDUCEPS, VRNDSCALEPS, VRSQRT14PS, VSCALEFPS	$R = op\ A$ $R = \check{P} ? (op\ A) : R$ $R = \check{P} ? (op\ A) : 0$
VADDPS, VANDPS, VANDNPS, VDIVPS, VMAXPS, VMINPS, VMULPS, VORPS, VSUBPS, VRANGEPS	$R = op\ A, B$ $R = \check{P} ? (op\ A, B) : R$ $R = \check{P} ? (op\ A, B) : 0$
VFMADD*PS, VFMSUB*PS, VFNMADD*PS, VFNMSUB*PS	$R = op\ R, A, B$ $R = \check{P} ? (op\ R, A, B) : R$ $R = \check{P} ? (op\ R, A, B) : 0$
VCMPPS	$\check{P} = op\ A, B$ $\check{P} = \check{Q} ? (op\ A, B) : 0$
VBLENDPS	$R = \check{P} ? A : B$

В таблице представлена семантика основных векторных инструкций для работы с вещественным программным контекстом, она представляет собой по сути плоские циклы.

Верно обратное – плоский цикл, записанный в предикатном виде, может быть векторизован путем замены скалярных инструкций векторными аналогами.

Далее рассматриваются преобразования кода, повышающие вероятность **автоматического** применения векторизации оптимизирующим компилятором, а также вручную выполняемые оптимизации кода с помощью **функций-интринсиков**.

Данные: «массив структур» vs «набор массивов»

```
1 struct Cell
2 {
3     double rho;
4     double u; double v; double w;
5     double p;
6     double rho_u; double rho_v; double rho_w;
7     double E;
8 };
9 Cell cells[N];
10
11 void d_to_u()
12 {
13     for (int i = 0; i < N; ++i)
14     {
15         double rho = cells[i].rho;
16         double u = cells[i].u;
17         double v = cells[i].v;
18         double w = cells[i].w;
19         double p = cells[i].p;
20
21         cells[i].rho_u = rho * u;
22         cells[i].rho_v = rho * v;
23         cells[i].rho_w = rho * w;
24         cells[i].E = 0.5 * rho * (u * u + v * v + w * w)
25                     + p / (GAMMA - 1.0);
26     }
27 }
```

организация данных в виде массива структур

```
1 double rhos[N];
2 double us[N]; double vs[N]; double ws[N];
3 double ps[N];
4 double rho_us[N]; double rho_vs[N]; double rho_ws[N];
5 double Es[N];
6
7 void d_to_u()
8 {
9     for (int i = 0; i < N; ++i)
10     {
11         double rho = rhos[i];
12         double u = us[i];
13         double v = vs[i];
14         double w = ws[i];
15         double p = ps[i];
16
17         rho_us[i] = rho * u;
18         rho_vs[i] = rho * v;
19         rho_ws[i] = rho * w;
20         Es[i] = 0.5 * rho * (u * u + v * v + w * w)
21                + p / (GAMMA - 1.0);
22     }
23 }
```

организация данных в виде набора массивов

Организация данных в виде набора массивов позволяет использовать векторные операции обращения к последовательным областям памяти и, таким образом, избавиться от медленных операций gather/scatter.

Расщепление гнезд циклов по условиям

```
1 | for (int k = 0; k < NZ; ++k)
2 | {
3 |     for (int j = 0; j < NY; ++j)
4 |     {
5 |         for (int i = 0; i < NX; ++i)
6 |         {
7 |             if (i == 0)
8 |             {
9 |                 // left boundary condition
10 |                ...
11 |            }
12 |
13 |            // rest code
14 |            ...
15 |        }
16 |    }
17 | }
```

гнездо циклов с условием внутри

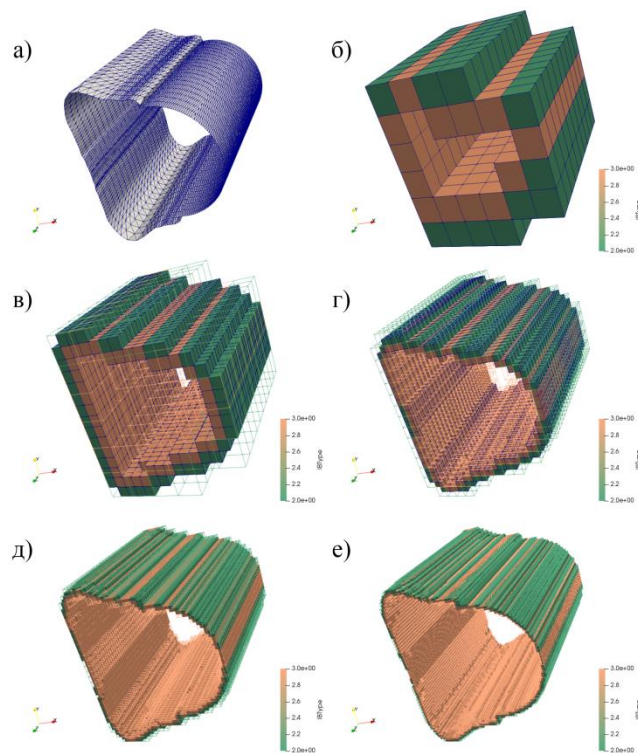
```
1 | for (int k = 0; k < NZ; ++k)
2 | {
3 |     for (int j = 0; j < NY; ++j)
4 |     {
5 |         // left boundary condition with i = 0
6 |         ...
7 |     }
8 | }
9 |
10 | for (int k = 0; k < NZ; ++k)
11 | {
12 |     for (int j = 0; j < NY; ++j)
13 |     {
14 |         for (int i = 1; i < NX; ++i)
15 |         {
16 |             // rest code
17 |             ...
18 |         }
19 |     }
20 | }
```

расщепление гнезда по условию

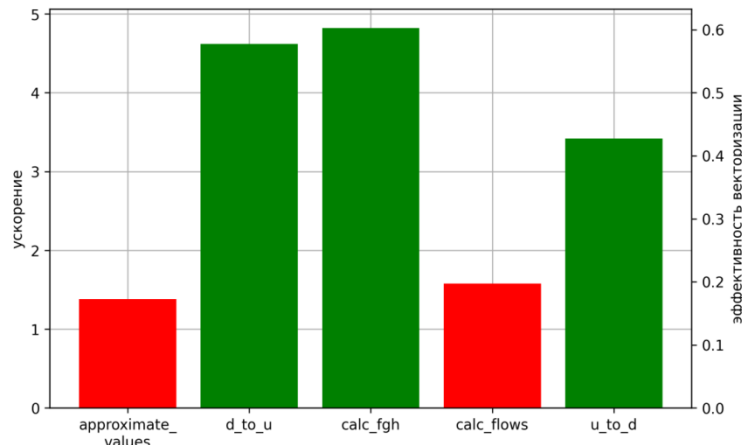
Наличие условий повышает количество операций передачи управления в цикле, что негативно сказывается на производительности векторного кода.

Расщепление циклов по константному условию позволяет уменьшить количество команд передачи управления и повысить производительность векторного кода.

Эксперимент по векторизации газодинамического решателя с методом погруженной границы



обстраивание поверхности декартовой
объемной сеткой для применения метода
погруженной границы



результаты векторизации газодинамического решателя

Автоматическая векторизация газодинамического решателя, использующего схему Стегера-Уорминга, с использованием метода погруженной границы (после замены схемы расположения данных и расщепления циклов по условию).

Плоские циклы продемонстрировали высокую эффективность векторизации (0,5), квазиплоские циклы векторизовались менее эффективно (0,2).

Вынос маловероятного региона из цикла

```
1 for (int i = 0; i < w; ++i)
2 {
3     block(i);
4
5     if (cond(i))
6     {
7         block_true(i); // prob. ~1.0
8     }
9     else
10    {
11        block_false(i); // prob. ~0.0
12    }
13 }
```

исходный цикл с маловероятным
регионом внутри

```
1 for (int i = 0; i < w; ++i)
2 {
3     block(i);
4     TMP[i] = cond(i);
5 }
6
7 for (int i = 0; i < w; ++i)
8 {
9     block_true(i) ? TMP[i];
10 }
11
12 if (TMP != 0x0)
13 {
14     for (int i = 0; i < w; ++i)
15     {
16         if (!TMP[i])
17         {
18             block_false(i);
19         }
20     }
21 }
```

вынос маловероятного региона

```
1 BLOCK();
2 TMP = COND();
3 BLOCK_TRUE() ? TMP;
4
5 if (TMP != 0x0)
6 {
7     for (int i = 0; i < w; ++i)
8     {
9         if (!TMP[i])
10        {
11            block_false(i);
12        }
13    }
14 }
```

векторизация вероятного кода

`block_true` – вероятная ветка с простым управлением, `block_false` – редкий сложно векторизуемый код.

Вместо исходного цикла создаются три новых.

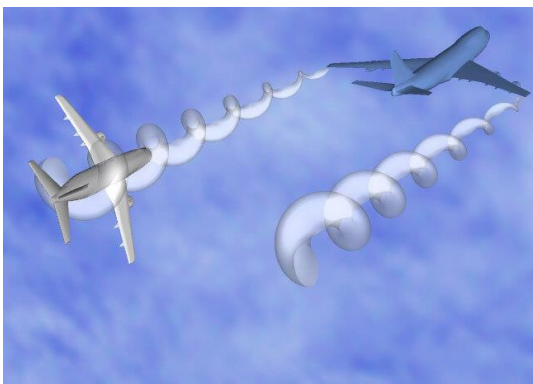
В первом цикле выполняется только блок `block(i)`, после которого условия `cond(i)` записываются в локальный массив булевых значений.

Во втором цикле на каждой итерации при истинном значении элемента `TMP[i]` выполняется ветка `block_true(i)`.

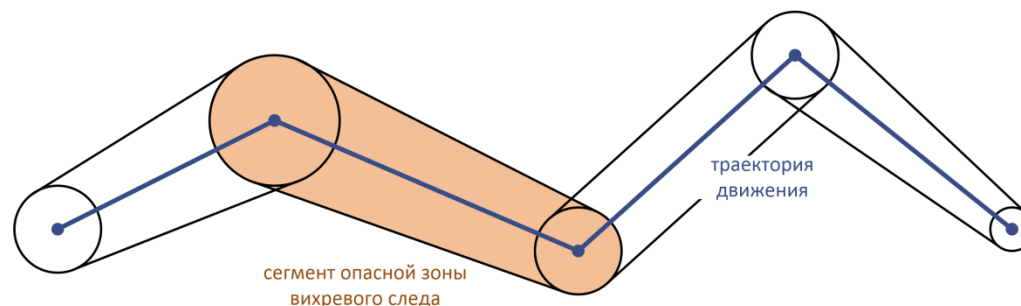
Третий цикл инкапсулирует в себе выполнение маловероятного кода при ложном `TMP[i]`.

Эксперимент по векторизации определения пересечения с опасной зоной ЛА

Задача обеспечения безопасности полетов ЛА. Во время полета ЛА генерирует вихревой спутный след, который может представлять опасность для других участников воздушного движения. Он может быть описан как совокупность окрестностей отрезков траектории движения. Для определения конфликта требуется решить задачу наличия пересечения траектории движения собственного ЛА в виде полупрямой с множеством окрестностей отрезков. Наличие конфликта является редкой исключительной ситуацией.



вихревого следа самолета



представление опасной зоны вихревого следа

Был поставлен численный эксперимент по определению точек пересечения траектории движения со спутными следами ЛА. Оптимизация выноса маловероятного региона позволила достичь пятикратного ускорения от **автоматической** векторизации.

Ручная векторизация с помощью интринсиков

```
01 void prefun(float &f, float &fd, float &p,  
02           float &dk, float &pk, float &ck)  
03 {  
04     float ak, bk, pratio, qrt;  
05  
06     if (p <= pk)  
07     {  
08         pratio = p / pk;  
09         f = G4 * ck * (pow(pratio, G1) - 1.0);  
10         fd = (1.0 / (dk * ck)) * pow(pratio, -G2);  
11     }  
12     else  
13     {  
14         ak = G5 / dk;  
15         bk = G6 * pk;  
16         qrt = sqrt(ak / (bk + p));  
17         f = (p - pk) * qrt;  
18         fd = (1.0 - 0.5 * (p - pk) / (bk + p)) * qrt;  
19     }  
20 }
```

скалярная функция

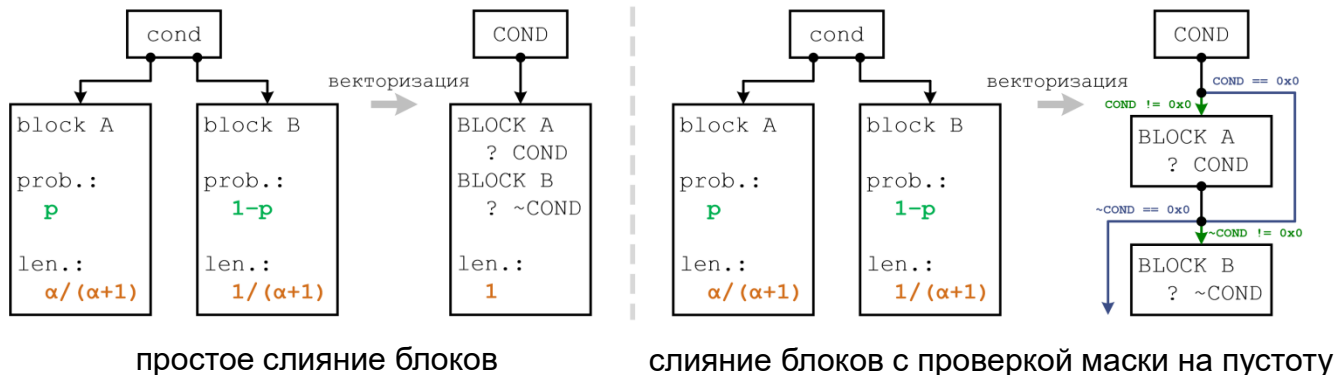
```
01 void prefun_16(__m512 *f, __m512 *fd, __m512 p,  
02               __m512 dk, __m512 pk, __m512 ck,  
03               __mmask16 m)  
04 {  
05     __m512 pratio, ak, bkp, ppk, qrt;  
06     __mmask16 cond, ncond;  
07  
08     // Conditions.  
09     cond = _mm512_mask_cmp_ps_mask(m, p, pk, _MM_CMPINT_LE);  
10     ncond = m & ~cond;  
11  
12     // The first branch.  
13     if (cond != 0x0)  
14     {  
15         pratio = _mm512_mask_div_ps(z, cond, p, pk);  
16         *f = _mm512_mask_mul_ps(*f, cond, MUL(g4, ck),  
17                                SUB(_mm512_mask_pow_ps(z, cond, pratio, g1), one));  
18         *fd = _mm512_mask_div_ps(*fd, cond,  
19                                _mm512_mask_pow_ps(z, cond, pratio, SUB(z, g2)),  
20                                MUL(dk, ck));  
21     }  
22  
23     // The second branch.  
24     if (ncond != 0x0)  
25     {  
26         ak = _mm512_mask_div_ps(z, ncond, g5, dk);  
27         bkp = FMADD(g6, pk, p);  
28         ppk = SUB(p, pk);  
29         qrt = _mm512_mask_sqrt_ps(z, ncond,  
30                                _mm512_mask_div_ps(z, ncond, ak, bkp));  
31         *f = _mm512_mask_mul_ps(*f, ncond, ppk, qrt);  
32         *fd = _mm512_mask_mul_ps(*fd, ncond, qrt,  
33                                FNMADD(_mm512_mask_div_ps(z, ncond, ppk, bkp),  
34                                SET1(0.5), one));  
35     }  
36 }
```

объединение 16 экземпляров функции в векторный аналог

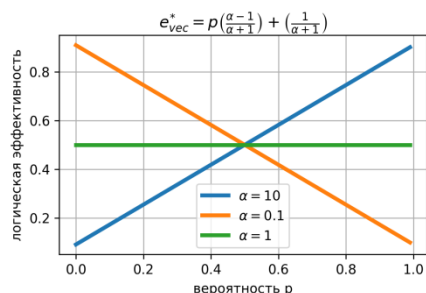
Функции-интринсики – специальные функции, которые раскрываются компилятором в инструкции AVX-512, либо последовательности инструкций, в некоторых случаях в библиотечные вызовы.

Интринсики позволяют без использования ассемблерных вставок создавать векторный код.

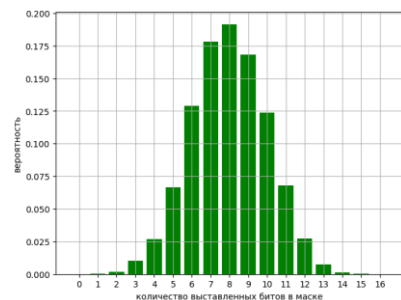
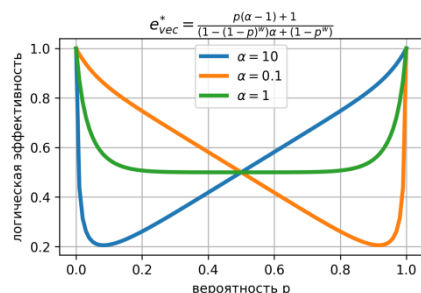
Слияние путей исполнения по условию



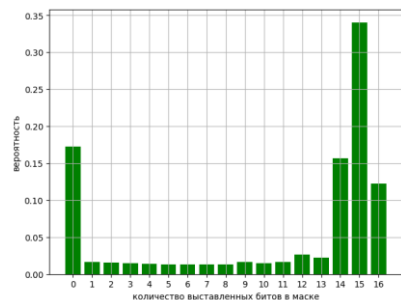
Слияние путей исполнения (ifconversion) позволяет объединить пути исполнения под предикатами.



логическая эффективность векторизации



диаграммы плотности векторной маски

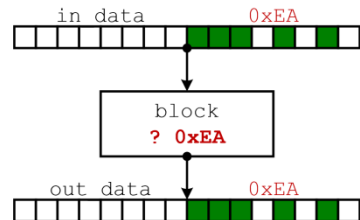


При возрастании количества условий эффективность векторизации падает экспоненциально.

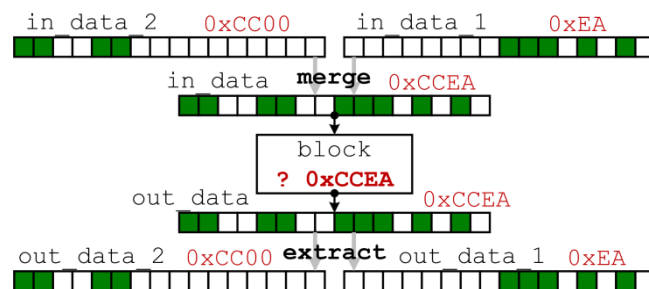
Профиль векторной маски в предположении независимости переходов существенно отличается от профиля на реальных приложениях,

Объединение и комбинирование масок

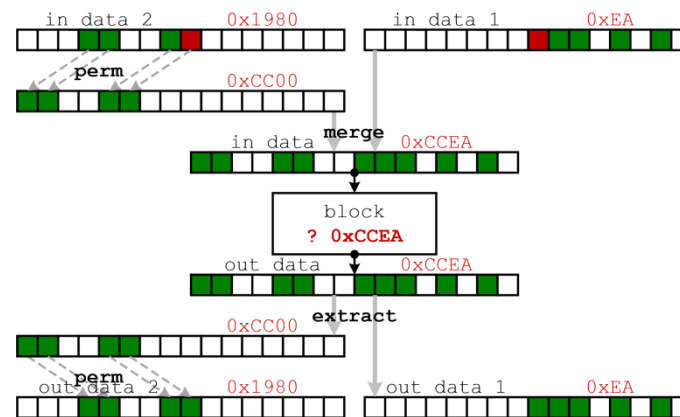
После разбиения цикла с количеством итераций больше w на последовательность плоских циклов, слияния путей исполнения в их телах и разбиения тел этих циклов на отдельные блоки получается последовательность одинаковых векторных блоков, исполняющихся под разными масками. В некоторых случаях соседние блоки можно объединить путем объединения и комбинирования масок.



исполнение векторного блока
под маской



объединение масок
соседних блоков



комбинирование масок
соседних блоков

Объединение масок – одновременное выполнение соседних векторных блоков, выполняющихся под непересекающимися масками.

Комбинирование масок – одновременное выполнение соседних векторных блоков, выполняющихся под пересекающимися масками с суммарным количеством единичных битов не более w .

Эти оптимизации позволяют повысить плотность векторных масок в коде, тем самым увеличивая логическую эффективность векторизации

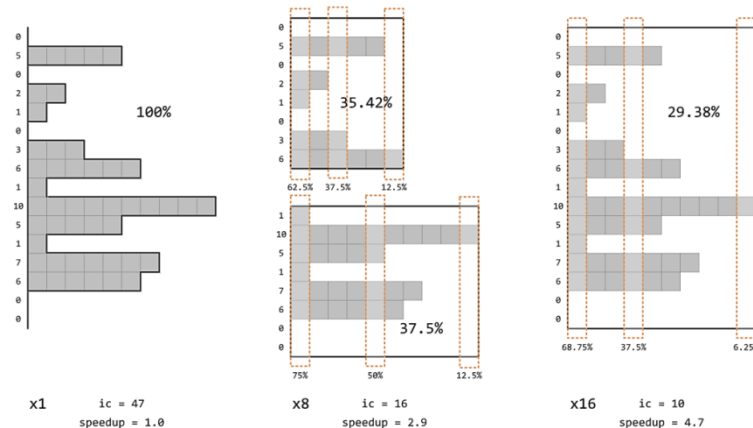
Векторизация гнезд циклов

```
// плоский цикл
for (i = 0; i < w; ++i)
{
    ...
    // внутренний цикл
    while (cond)
    {
        block
        cond = new_cond
    }
    ...
}
```

векторизация →

```
...
// векторный блок
while (COND != 0x0)
{
    BLOCK ? COND
    COND &= NEW_COND
}
...
```

схема векторизации гнезда «плоский цикл / внутренний цикл»



при увеличении ширины векторизации потери, связанные с неравномерности количества итераций внутреннего цикла, возрастают

$I(i)$ – количество итераций внутреннего цикла на i -ой итерации в скалярной версии.
 I_v – количество итераций внутреннего цикла в векторной версии $I_v = \max_{i=0}^{w-1} I(i)$.

При векторизации гнезда «плоский цикл / внутренний цикл» верна оценка $e_{vec}^* \leq e_{vec}^I$, где $e_{vec}^I = \frac{\sum_{i=0}^{w-1} I(i)}{I_v w}$

При этом, если $I_v - \min_{i=0}^{w-1} I(i) \leq \varepsilon$, то $e_{vec}^I \geq 1 - \varepsilon / I_v$.

Оценка верна в предположении идеальной векторизации итерации внутреннего цикла и постоянном количестве операций на итерации внутреннего цикла.

Параметр ε – показатель **неравномерности** количества итераций внутреннего цикла. Чем выше этот показатель, тем больше потери при векторизации.

Векторизация гнезд циклов

Рассмотрен программный контекст разного вида.

1. **Постоянное количество итераций.** Для него $\varepsilon = 0$, $e_{vec}^I = 1$.

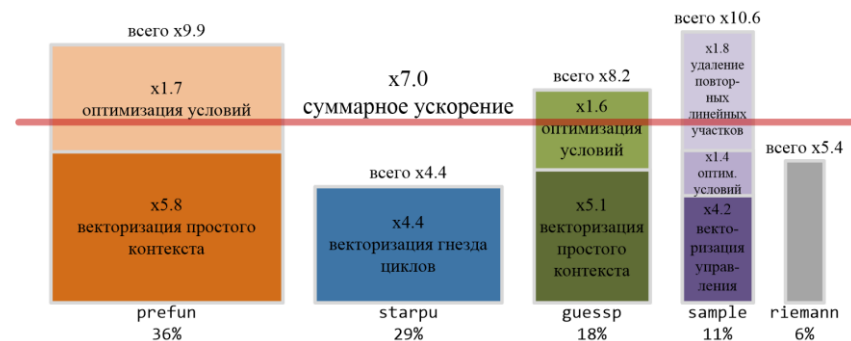
Случай имеет место при векторизации нахождения пересечения поверхностной сетки с подложкой в методе погруженной границы (векторизованная версия продемонстрирована ускорение в 6,7 раз).

2. **Непостоянное количество итераций.** Для него ε мал, и e_{vec}^I близок к единице.

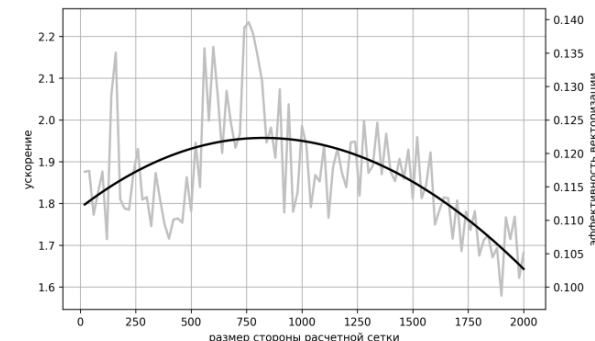
Случай имеет место при векторизации решения нелинейного уравнения из реализации точного римановского решателя (ускорение около 4,4 раза).

3. **Нерегулярное количество итераций.** Для него ε сравним с I_v , и e_{vec}^I достаточно мал.

Случай имеет место при векторизации задачи декомпозиции неструктурированной расчетной сетки (ускорение менее 2,0 раз).

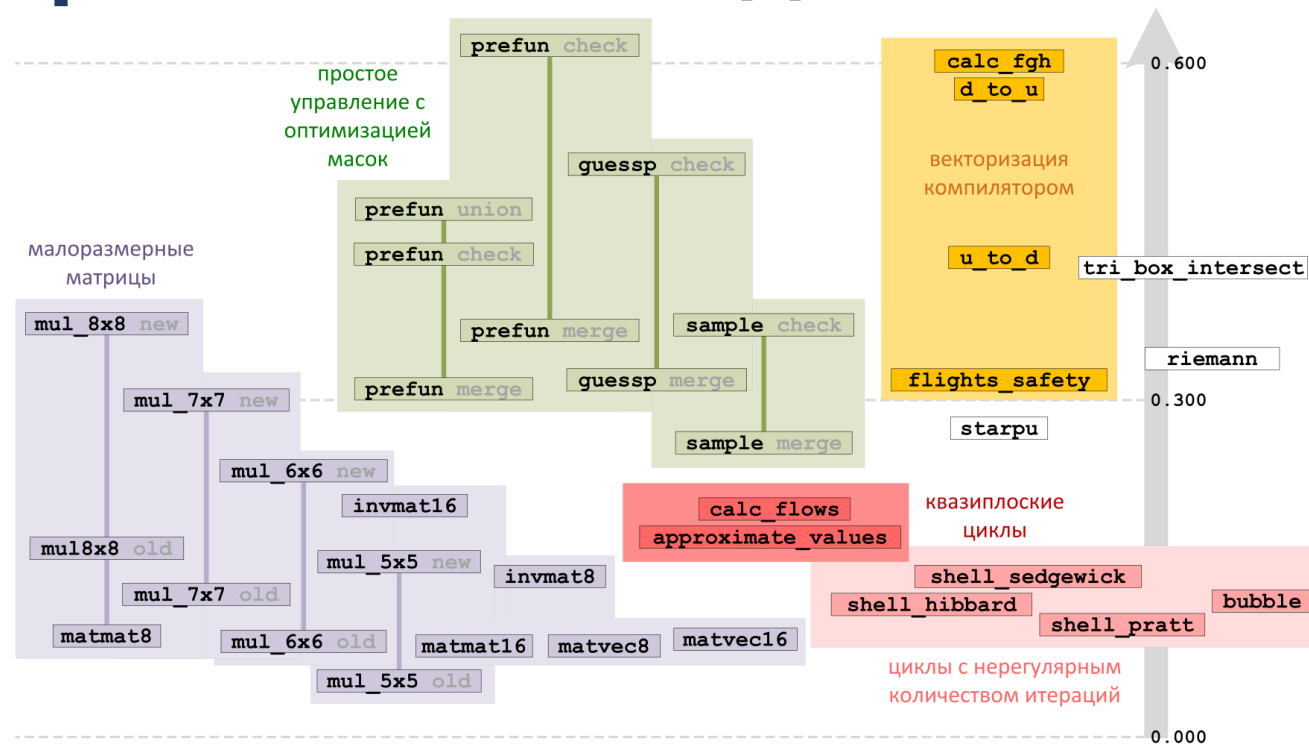


ускорение отдельных функций точного римановского решателя



ускорение декомпозиции сетки

Эффективность векторизации программного контекста различного вида



Карта эффективности векторизации программного контекста

Наибольшую эффективность векторизации демонстрируют плоские циклы с **арифметикой** и **простым управлением** с применением **проверки масок на пустоту**, а также **объединения масок**.

Наименьшую эффективность векторизации демонстрируют **квазиплоские** циклы, а также гнезда циклов с **нерегулярным количеством итераций**.

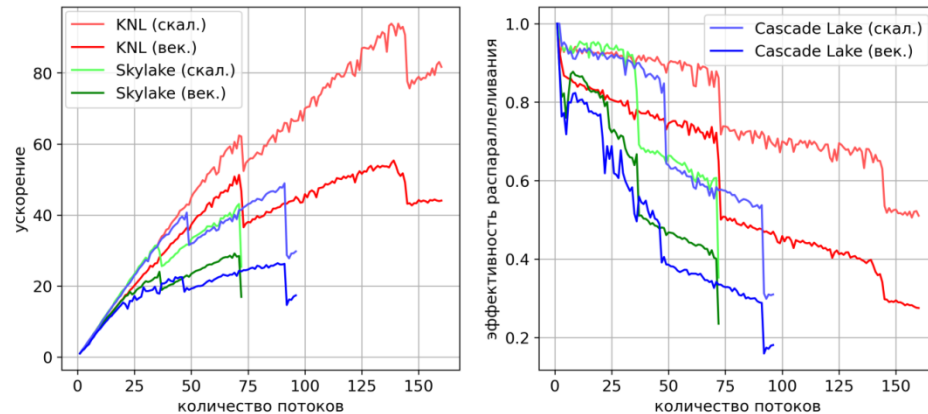
Масштабирование скалярных и векторных вычислений

Эксперимент по масштабированию скалярного и векторного кода на примере точного римановского решателя на микропроцессоре Intel Xeon Phi KNL.

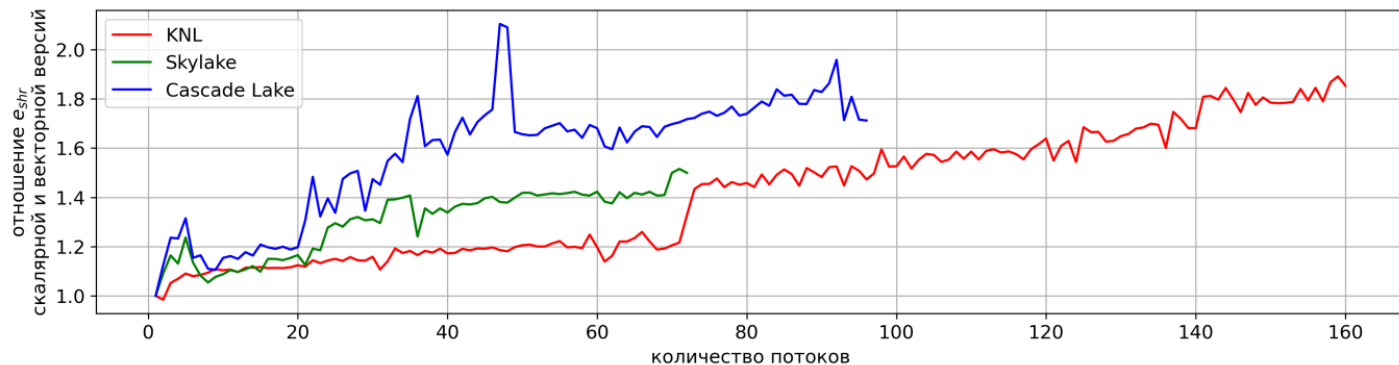
Для эксперимента использована скалярная и векторная версия решения задачи Римана о распаде разрыва.

Векторный код демонстрирует масштабируемость хуже скалярного кода.

График отношения эффективности соответствует нижней границе эффективности векторизации, ниже которой не имеет смысла векторизовать программный код.



Графики масштабирования скалярного и векторного кода



Отношение эффективности масштабирования скалярного и векторного кода

Заключение

1. Векторизация вычислений позволяет кратно повысить производительность расчетов. Набор инструкций AVX-512 обладает рядом особенностей, позволяющих повысить эффективность векторизации.
2. Представление вычислений в виде композиции плоских циклов расширяет возможности применения векторизации.
3. Простые рекомендации по компоновке кода повышают вероятность применения векторизации оптимизирующим компилятором.
4. Плоский цикл с телом практически произвольной формы, записанным в предикатном виде, может быть векторизован путем замены скалярных операций векторными аналогами автоматически с помощью оптимизирующего компилятора либо с помощью функций-интринсиков.
5. Основными причинами снижения эффективности векторизации являются:
 - 1) низкая плотность векторных масок в коде, обусловленная большим количеством **операций передачи управления**, а также **нерегулярностью количества итераций** в гнездах циклов;
 - 2) появление невыровненных обращений в память и медленных операций **gather/scatter** из-за нарушения требований плоских циклов.
6. В исследовании продемонстрировано применение векторизации к программному контексту различного вида.

Список источников

Cebrian J., Natvig L., Lahre M. *Scalability analysis of AVX-512 extensions.* // The Journal of Supercomputing, 2020, Vol. 76, P. 2082-2097. DOI: 10.1007/s11227-019-02840-7

Intel 64 and IA-32 architectures software developer's manual. Combined volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4. // Order number: 325462-087US, march 2025.

Intel Intrinsics Guide, URL: <https://alouettesu.github.io/Intrinsics/> (дата обращения 24.11.2025)

Jeffers J., Reinders J., Sodani A. *Intel Xeon Phi processor high performance programming: Knights Landing edition.* // Morgan Kaufmann, 2016, 662 P.

Laukemann J., Hammer J., Hager G., Wellein G. *Automatic throughput and critical path analysis of x86 and ARM assembly kernels.* // Proc. IEEE/ACM PMBS, 2019, P. 1–6. DOI: 10.1109/PMBS49563.2019.00006

Kusswurm D. *Modern parallel programming with C++ and Assembly Language. X86 SIMD development using AVX, AVX2, and AVX-512.* // CA, Apress Berkeley Publ., 2022, 633 P. DOI: 10.1007/978-1-4842-7918-2

Волконский В., Окунев С. *Предикатное представление как основа оптимизации программы для архитектур с явно выраженной параллельностью.* // Информационные технологии, 2003, № 4, С. 36–45.

Savin G., Shabanov B., Rybakov A., Shumilin S. *Vectorization of flat loops of arbitrary structure using instructions AVX-512.* // Lobachevskii Journal of Mathematics, 2020, Vol. 41, No. 12, P. 2566-2574. DOI: 10.1134/S1995080220120331

Muchnick S. *Advanced compiler design and implementation.* // Morgan Kaufmann Publishers, 1997.

Krzikalla O., Wende F., Hohnerbach M. *Dynamic SIMD vector lane scheduling.* // ISC High Performance 2016: High Performance Computing, Lecture Notes Computer Science, 2016, Vol. 9945, P. 354–365. DOI: 10.1007/978-3-319-46079-6_25

Shabanov B., Rybakov A., Shumilin S. *Vectorization of high-performance scientific calculations using AVX-512 instruction set.* // Lobachevskii Journal of Mathematics, 2019, Vol. 40, No. 5, P. 580-598. DOI: 10.1134/S1995080219050196



ICAM 2025

Спасибо за внимание!